Medicine Cold Chain Integrity System using Blockchain and Smart Contracts

A Course Project Report

Submitted by

APPALA SHANMUKHA SAI SRI CHARAN (CB.EN.U4ECE23206)
CHITTESH S (CB.EN.U4ECE23212)

in the course

Blockchain Technology and its Applications

Semester VII

offered by the

Department of Computer Science and Engineering



Department of Computer Science and Engineering,
Amrita School of Engineering,
Amrita Vishwa Vidyapeetham,
Coimbatore – 641112

October 2026

1. Introduction

Vaccines, insulin and many other medicines have to be kept between 2 °C and 8 °C from the time they leave the factory until they reach the patient. If the temperature goes outside this range the medicine can lose its effect, and this cannot be seen by looking at the package. In most supply chains, transfer records and temperature logs are kept by each company in its own system. A distributor that had a cooling failure can simply edit its log, and the pharmacy or the patient has no way to find out.

In this project we built a decentralized application (dApp) in which the important records of a medicine batch are stored on the Ethereum blockchain. Two smart contracts written in Solidity decide who can add a batch, in what order it can be sold, and whether each temperature reading is inside the allowed range. Data written to the blockchain cannot be changed later, so any participant, including the customer holding the medicine, can check the full history of a batch from its QR code. The aim was to record each batch with its temperature limits on a public blockchain, to enforce the order Manufacturer → Wholesaler → Pharmacy in contract code, to accept readings only from an authorized logger, and to give a simple web interface for QR verification and auditing.

2. System Design

The overall structure of the system is shown in Fig. 1. Users work with a React web application. The application has no backend server or database of its own; it reads from and writes to the blockchain through the MetaMask wallet, which signs every transaction with the user's private key. All the business rules are inside the smart contracts deployed on the Sepolia test network. Reading data, for example with getMedicine(), is a free call, while functions that change data, such as addMedicine() or recordTemperature(), are sent as transactions and need Sepolia ETH for gas.

Five user roles are defined in the registration contract, and their tasks are listed in Table 1. The logger is not a user account. It is an Ethereum address, normally belonging to an IoT gateway in a truck or a warehouse, which the manufacturer authorizes to send readings for one particular batch.

Table 1 Participants and their responsibilities

Participant	Responsibility in the system
Manufacturer	Registers a batch with its temperature limits and authorizes a logger wallet
Wholesaler	Buys batches that are in the Manufactured state
Distributor	Tracks batches and checks their cold-chain status and audit trail
Pharmacy	Buys batches from the wholesaler once they are InStock
Customer	Verifies a medicine by its QR code and views its history
Logger (wallet)	Sends temperature readings for the batch it is authorized for

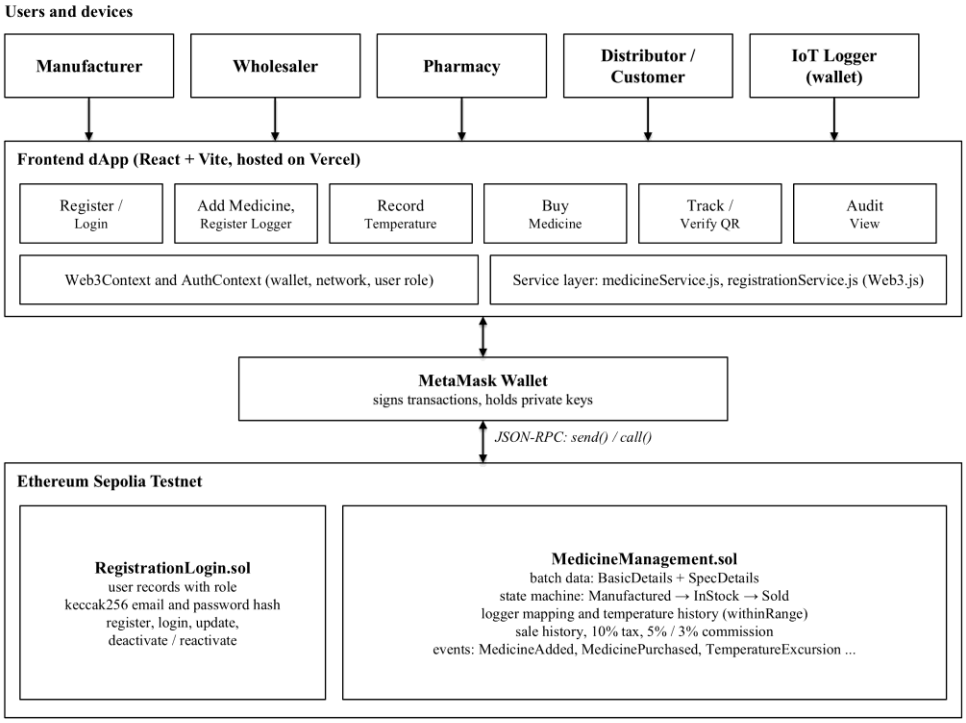


Fig. 1 Block diagram of the Medicine Cold Chain Integrity System

3. Smart Contract Design

3.1 Registration Contract

RegistrationLogin.sol stores each user under the keccak256 hash of the email address, and the password is also kept only as a keccak256 hash. The register() function checks that the name and email are given, the phone number has 10 digits, the password has at least 8 characters, the role is one of the five valid roles and the terms have been accepted. The contract also supports changing the address or password, deactivating an account, and reactivating it after 30 days.

3.2 Batch Registration and Lifecycle

MedicineManagement.sol (file AddNewMedicine.sol) keeps every batch in two structures. *BasicDetails* holds the identity of the batch: medicine ID, name, type, strength, batch number, storage conditions, and the manufacturer's ID and wallet address. *SpecDetails* holds the manufacture and expiry dates, price, quantity, current state, and the minimum and maximum temperature. When a batch is added, a 10% tax is added to the base price and the batch starts in the Manufactured state. The purchase rules are given in Table 2.

Table 2 Purchase rules enforced by purchaseMedicine()

Buyer	Required state	Price update	State after purchase
Wholesaler (buyerType 1)	Manufactured	+5% commission	InStock
Pharmacy (buyerType 2)	InStock	+3% commission	Sold when quantity reaches 0, else InStock

In both cases the contract also checks that enough quantity is left and that the ETH sent covers the unit price multiplied by the quantity. Every sale is stored in the saleHistory array and announced through the MedicinePurchased and SaleHistoryRecorded events. Since a pharmacy can buy only from the InStock state, the wholesale stage cannot be skipped.

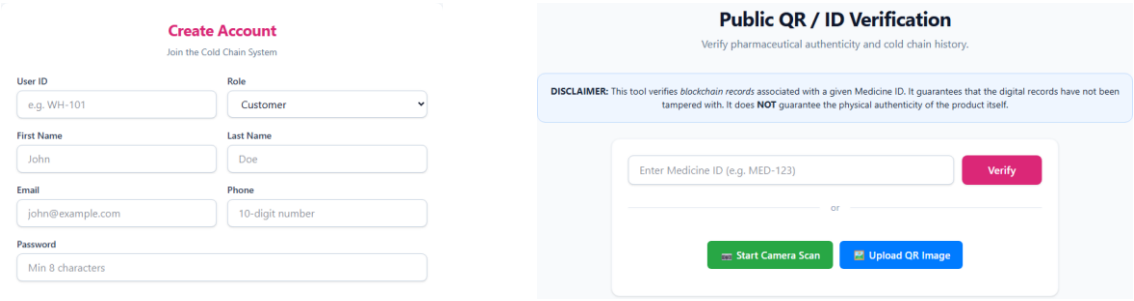
3.3 Cold-Chain Monitoring

The manufacturer calls registerLogger() to link a logger address to a batch. The onlyManufacturer modifier allows this only from the wallet that created the batch and only with the matching manufacturer ID. After that, recordTemperature() accepts a reading only when msg.sender is the registered logger. The contract compares the value with tempMin and tempMax, with both limits included, and stores the block timestamp, the temperature and a withinRange flag. A TemperatureRecorded event is emitted for every reading and a TemperatureExcursion event is emitted when the value is out of range. The function getColdChainStatus() returns two flags, hasReadings and isValid. No function removes readings, so a single excursion marks the batch as invalid for good.

4. Frontend and Deployment

The frontend was developed with React and Vite, using React Router for its 11 pages and Tailwind CSS for styling. Web3Context keeps track of the connected wallet and warns the user when MetaMask is not on Sepolia (chain ID 0xaa36a7), and AuthContext remembers the logged-in user for each wallet. All contract calls go through two service files, medicineService.js and registrationService.js, which use Web3.js with the contract ABIs. The contract addresses are read from environment variables, and the application is hosted on Vercel at https://cold-chain-track.vercel.app.

The menu changes with the role. A manufacturer sees Add Medicine and Register Logger, while wholesalers and pharmacies see the Buy page, which lists only the batches they are allowed to buy. The Track page shows the batch details, a QR code generated with qrcode.react, a Chart.js graph of the readings with the two limit lines, and a table of all readings. The Verify page (Fig. 2b) takes a medicine ID typed by the user, scanned with the camera or read from an uploaded QR image. The Audit page rebuilds the history of a batch from the contract events in block order, with a link to each transaction on Etherscan. To stay within the block-range limit of the RPC provider, it first searches only the last 9,000 blocks.



(a) Registration with role selection (b) Public QR / ID verification

Fig. 2 Screenshots of the deployed application

5. Testing

The contracts were deployed through Remix IDE and tested manually on Sepolia together with the frontend. The following cases were checked:

- registration and login for every role, adding a batch and authorizing a logger wallet;
- readings for a 2–8 °C batch: 2 °C and 8 °C were accepted and 10 °C was recorded as an excursion;
- a wholesaler purchase (Manufactured → InStock) and pharmacy purchases until the batch became Sold;
- a reading sent from a wallet other than the logger, which the contract rejected; and
- the Audit page listing all events of a batch in the correct order.

No automated test scripts were written for the contracts; all the checks above were done by hand.

6. Limitations and Future Work

While going through the code we noted a few points that need to be fixed before real use. addMedicine() and purchaseMedicine() do not check the caller's role on-chain; this check is done only in the React pages. login() is sent as a transaction, so the password is visible in the transaction data even though only its hash is stored. The ETH paid for a purchase stays in the contract, as there is no function to pay the seller or refund extra payment. The Expired state is never set automatically and the expiry date is not checked at purchase. Inventory is tracked for the whole batch rather than per owner, and the system has to trust that the logger device reports the true temperature. Future work includes mapping wallet addresses to roles inside the contract, wallet-signature login in place of passwords, automatic expiry handling, and connecting a real IoT temperature sensor to the logger wallet.

7. Conclusion

The project shows that a public blockchain can hold cold-chain records that no single party is able to change. The smart contracts enforce the order of sale, accept readings only from the authorized logger and decide on-chain whether each reading is safe. With the QR verification and audit pages, a pharmacy, a regulator or a patient can check a batch without depending on the records kept by the companies that handled it.